# Inheritance, Abstract Classes, and Interfaces

CSCI 2113 — Lab 4: one hierarchy for drivers and riders

---

Alex (Aizierjiang Aiersilan)
Lab Lead, Teaching Assistant

September 20, 2026

The George Washington University
CSCI 2113 - Software Engineering | Fall 2026

© Aizierjiang Aiersilan. Free distribution and open-access.

# Outline


1. Where Lab 4 Fits
2. Constructors and `super(...)`
3. Inheriting and Overriding
4. Abstract Classes
5. Interfaces
6. A Console That Loops
7. Pulling `User` Out of `Driver` and `Rider`
8. `RideShare`: Data Without I/O
9. The Assignment
10. Wrap-up
11. Acknowledgement


ng Aiersman | CSCI 2113

# Where Lab 4 Fits

ng Aiersilan | CSCI 2113 | F

# You are about to write the same class twice

Lab 3 left you with **Driver** and **Vehicle**: a private `nextId` counter, an `id` assigned in the constructor, get methods, `toString()` and `equals()`.

Today the app gains a **Rider** — a name, a rating, and an id.

- The counter, the id, the name, the rating and their get methods appear in *both* files.
- Each class counts from **1** on its own, so a driver and a rider can both be #1.

**Today's question**

Two classes, one shared idea. How do you say that **once** — and make Java enforce it?

# Four mechanisms, one ride-share app

1. **Inheritance** — `extends`, and the `super(...)` call you never wrote.
2. **Overriding** — replacing a parent's method, and `@Override` as your safety net.
3. **Abstract classes** — a class you may extend but never build.
4. **Interfaces** — a promise two unrelated classes can both keep.

💡 All four land in one place: an abstract **User** that **Driver** and **Rider** share — driven from a console menu.

ng Aiersilan | CSCI 2113

# Before you type: which Java this lab assumes

The lab writes `IO.println(...)` and `IO.readln("Name: ")`. **IO** is a real class — `java.lang.IO`, `public final`, new in Java **25** [7, 5]. It lives in `java.lang`, which every file imports implicitly, so no `import` line is needed [3].

```
static void println(Object obj)   static String readln()
static void println()             static String readln(String prompt)
static void print(Object obj)
```

- ⚠ One Activity step puts a statement *before* `super(...)`. That is **flexible constructor bodies**, also standard from Java **25** [6, 8].
- On an older JDK: use `System.out.println` / `Scanner`, and `super(...)` must be the **first** statement.

## Constructors and `super(...)`

ng Aiersilan | CSCI 2113 | F

# Step 1: write the prediction down first

A fresh directory — none of this is part of the ride-share app.

```java
public class Parent {
    public Parent() {
        IO.println("Parent()");
    }
}
```

```java
public class Child extends Parent {
    public Child() {
        IO.println("Child()");
    }
}
```

```java
public class Main {
    public static void main(String[] args) { Child c = new Child(); }
}
```

⚠ Write your prediction down, then run **Main**. Guessing in your head does not count.

Aiersilan | CSCI 2113

# The call you never wrote

```
Parent()
Child()
```

**Child**'s constructor never mentions **Parent** — yet `Parent()` ran *first*.

If a constructor body contains **no** explicit `this(...)` or `super(...)` at all, the compiler puts `super();` at the top for you [3].

**A subclass constructor never builds the whole object.** It finishes one the parent started.

CSCI 2113

# Step 2: give the parent something to demand

Change **Parent**'s constructor to take an `int x` and print `"Parent(" + x + ")"`. Compile:

```
Child.java:2: error: constructor Parent in class Parent cannot be applied to given types;
    public Child() {
                   ^
  required: int
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

- The rule being violated: the inserted `super()` needs a no-argument constructor in **Parent** — and writing `Parent(int)` means Java no longer supplies one.
- The error is reported in **Child**, at a line that looks innocent. ⚠ ✗ The cause is in the other file.

# Steps 3–5: put `super(7)` in, then move it

**`super(7);` first**

```
public Child() {
    super(7);
    IO.println("Child()");
}

Parent(7)
Child()
```

**`super(7);` moved down**

```
public Child() {
    IO.println("Child()");
    super(7);
}

Child()
Parent(7)
```

Statements before `super(...)` are the **prologue**. They run first, and they may not *observe* the object being built — no field reads, no instance methods, no handing `this` to anything [6]. (Plain assignments to this class's own fields *are* allowed.) Then put `super(7)` back on the first line.

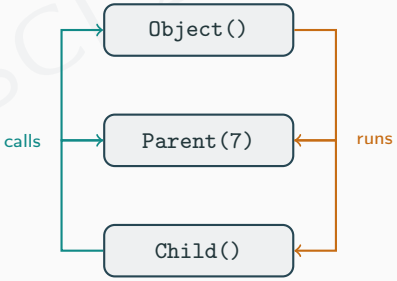
# Construction climbs up, then runs down

Every constructor contains **exactly one** of:

- `this(...)` — another constructor of the *same* class, or
- `super(...)` — explicit or compiler-inserted.

So the chain always climbs to **Object**. Then the bodies run from the top down: the parent is fully built before the child's first line.

That is why a child may safely use what the parent set up — and why the parent must never call down into a child.



# Inheriting and Overriding

ng Aiersilan | CSCI 2113 | F

# Steps 1–2: a method the child never wrote

```
// in Parent
public void greet() { IO.println("Parent.greet()"); }

// in Main
Parent p = new Child();
p.greet();
```

```
Parent(7)
Child()
Parent.greet()
```

**Child** declares no `greet()`; it **inherited** **Parent**'s. And note the first line: a variable declared **Parent** happily holds a **Child** — a **Child** *is a* **Parent** [4].

ng Aierqilan | CSCI 2113

# Step 3: the typo nobody reports

Add `gret()` — misspelled on purpose — to **Child**:

```
public void gret() { IO.println("Child.greet()"); }

Parent(7)
Child()
Parent.greet()
```

Nothing changed, and nothing was reported. To the compiler, `gret` is simply a **new method** that **Child** happens to have. It replaces nothing.

⚠ ✖ This is the bug class the next slide exists to kill.

CSCI 2113

# Step 4: `@Override` is a question you ask the compiler

```
@Override
public void gret() { IO.println("Child.greet()"); }
```

```
Child.java:7: error: method does not override or implement a method from a supertype
```

`@Override` does not *make* a method override anything. It asks the compiler to check that it already does [3]. Fix the name to `greet` and the last line becomes `Child.greet()`.

Habit worth forming today: **every override carries `@Override`** [1].

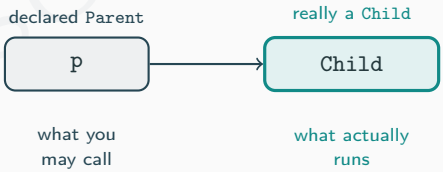
ng Aiersilan | CSCI 2113

# Two types, two different jobs

`Parent p = new Child();` gives you *two* types at once.

- The **declared type** (**Parent**) decides what you may call — checked at compile time.
- The **object's type** (**Child**) decides whose body runs — resolved at run time [3].

That second half is **polymorphism**, and it is the whole reason today's refactoring pays off: one loop over **User**, and each object answers for itself.




CSCI 2113

# Step 5: `super.greet()` — extend, don't erase

```
@Override
public void greet() {
    super.greet();                    // the version you just replaced
    IO.println("Child.greet()");
}
```

```
Parent(7)
Child()
Parent.greet()
Child.greet()
```

- `super.greet()` names **Parent**'s body **specifically** — it is the one call that does *not* dispatch on the object.
- Put it last instead and the two lines swap. You choose: before, after, or not at all.

# Abstract Classes

ng Aiersilan | CSCI 2113 | F

# Step 1: a method with no body

```
// in Parent
public void greet() { IO.println("Parent.greet() from " + label()); }
public abstract String label();
```

```
Parent.java:1: error: Parent is not abstract and does not override abstract method label() in Parent
```

**Parent** now promises a `label()` that nobody has written — and a class you can `new` may not contain a hole.

Read the message literally: *this class* does not implement its *own* abstract method.

ng Aiersilan | CSCI 2113

# Step 2: the error moves downhill

Declare `public abstract class Parent`. Recompile:

```
Child.java:1: error: Child is not abstract and does not override abstract method label() in Parent
```

The hole did not disappear — it was **inherited**. Every subclass either fills it, or is declared `abstract` itself and passes it further down [3].




ng Aiersten | CSCI 2113

# Step 3: the parent calls a method only the child has

```
// in Child
@Override
public String label() { return "Child"; }

Parent(7)
Child()
Parent.greet() from Child
Child.greet()
```

**Parent** wrote `greet()` without knowing what a label *is*. **Child** supplied the one piece that varies.

**The shape to remember**

An abstract class is a finished algorithm with a labelled gap. The parent fixes **what happens**; the subclass fills in **the part that differs**.

# Step 4: you may name it, you may not build it

```
Parent q = new Parent(3);       // delete this line after you read the error
```

```
Main.java:5: error: Parent is abstract; cannot be instantiated
```

The rule: **an abstract class has no instances** [3]. It exists to be extended.

### Still perfectly legal

```
Parent p = new Child();
Parent[] all = new Parent[2];
```

### Rejected

```
new Parent(3);
```

The *type* is usable; the *constructor* is not. Remember this when **User** becomes abstract later today.

# Interfaces

ng Aiersilan | CSCI 2113 | F

# Step 1: a promise with nothing behind it

```
public interface Describable {
    String describe();
}

public class Unrelated implements Describable {
}
```

```
Unrelated.java:1: error: Unrelated is not abstract and does not override abstract method describe() in Describable
```

Same sentence as the abstract class, different source. An interface method written without a body is implicitly `public and abstract` [3] — so your `describe()` must be declared `public` too.

ng Aiersman / CSCI 2113

# Steps 2–4: one array, two unrelated classes

```
public class Child extends Parent implements Describable { ... }

Describable[] things = new Describable[2];
things[0] = new Child();
things[1] = new Unrelated();
for (int i = 0; i < things.length; i++) { IO.println(things[i].describe()); }
```

```
Parent(7)
Child()
a Child object
an Unrelated object
```

**Child** and **Unrelated** share no superclass but **Object** — and one array holds both. A class has **one** parent; it may keep many promises.

ng Aierxilan | CSCI 2113

# Step 5: the array's type sets the menu

```
things[0].greet();      // things[0] really IS a Child
```

```
Main.java:6: error: cannot find symbol
  symbol:   method greet()
  location: interface Describable
```

The object has `greet()`. The **declared type** does not — and the compiler only offers what **Describable** declares.

💡 The same rule as two sections ago, wearing different clothes:
the declared type decides what you may call.

ng Aiersilan | CSCI 2113

# `extends` or `implements`?

### `extends` a class

- Exactly **one** parent.
- Inherits **fields and code**.
- "is a kind of" — **Driver** *is a* **User**.

### `implements` an interface

- As *many* as you like.
- Declares **a contract**, holds no instance state.
- "can do" — cuts *across* the hierarchy.

💡 Rule of thumb: shared *data and code* → a class; a shared *ability* → an interface [1, 2].

# A Console That Loops

ng Aiersilan | CSCI 2113 | F

# Read a line, then make it a number

```
public class Console {
    public static void main(String[] args) {
        String input = IO.readln("Enter a number: ");
        int number = Integer.parseInt(input);
        IO.println("You entered " + number);
    }
}
```

Now type abc:

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"
```

Standard input is always text. `Integer.parseInt` is the conversion, and it is where bad input shows up.

⚠ *Not* this lab's problem — we assume well-formed input, and handle it properly in a later lab.

ng Aiersilan | CSCI 2113

# The loop every console program has

```
while (true) {
    // show the menu
    // gather user input
    // process the input
    if (input.equals("q")) {
        break;
    }
}
```

**Three beats, forever**

**show → read → process**

The same skeleton a game loop has. `break` is the only way out.

⚠ ✗ Compare strings with `.equals(...)`, never with `==`. `==` asks whether two references point at the same object, which is not the question you mean to ask.

ng Aiersion | CSCI 2113

# The menu, and the two fields behind it

```
---
Main Menu
(1) Add a name
(2) List names
(q) Quit
Your choice:
```

```
private static String[] names = new String[100];
private static int nameCount = 0;
```

`names` has **100** slots; `nameCount` says how many are *real*. Loop to `nameCount`, never to `names.length`.

- Prompt changes to `"Your choice:  "` — and you stop parsing it. A menu choice is a **label**, not a number; multiplying it would be nonsense.
- `1` → read a `"Name:  "` and store it (remember to bump the count). `2` → print slots **0** to `nameCount` − **1**. Anything else → do nothing.

ng Aiersuan / CSU 2113

# Pulling `User` Out of `Driver` and `Rider`

ng Aiersilan | CSCI 2113 | F

# Write Rider, then put the two files side by side

**Rider** follows your Lab 3 **Driver** exactly: a private `static int nextId`, an `id` assigned in the constructor, get methods, and

`Rider #1 Ana`

**Copied into both files**

`nextId`, `id`, `getId()`, `name`, `getName()`, `rating`, `getRating()`, `setRating(...)`

**And worse**

Each class counts from **1** on its own — so Driver #1 and Rider #1 are two different accounts with the same number.

# Pull the shared parts *up*

```
public class User {
    public User(String name, double rating) {   // stores name, validates
        ...                                     // rating, assigns next id
    }
}
public class Driver extends User {
    public Driver(String name, int zone, boolean available,
                  double rating, Vehicle vehicle) {
        super(name, rating);                    // the explicit call
        ...
    }
}
```

Move `nextId`, `id`, `name`, `rating`, their get methods and `setRating` into **User**; delete them from **Driver** and **Rider**. Each subclass constructor replaces the assignments you deleted with one `super(name, rating)` call. **Driver**'s four-argument constructor still chains to the five-argument one with `this(...)`, exactly as in Lab 3.

ng Alrasilan / CSCI 2113

# `private` means private — even from your own child

**Driver**'s `toString()` and `equals()` still say `this.name` and `this.rating`. Compile:

```
error: name has private access in User
```

Inheritance gives **Driver** the **behaviour User** exposes — not a key to its locked drawers. Replace the references with `getName()` and `getRating()`.

**Driver** no longer holds the name. **User** does, and **User** keeps it private. That is the point, not an inconvenience.

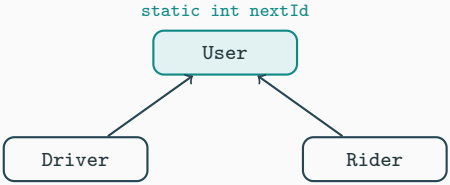
# One counter, one hierarchy

Construct a driver, a rider, and a second driver, then print their ids:

```
1
2
3
```

`nextId` now lives in **User**, and *every* **Driver** and **Rider** construction runs **User**'s constructor on its way up.

So there is **one** sequence for the whole system, and ids no longer collide across classes.




ng Aiersilan / CSCI 2113

# Nobody is *just* a user

```
public abstract class User { ... }
```

Every account in this system is a **Driver** or a **Rider**. The application never needs a plain **User** object — so say that in the code, and let the compiler enforce it.

✔ `User[] all = ...;` still works.

✔ **Driver** and **Rider** still compile.

✖ `new User("Ana", 5.0)` is now rejected.

The rest of the code does not change at all. An abstract class removes a possibility you were never using.

# `RideShare`: Data Without I/O

ng Aiersilan | CSCI 2113 | F

# A class that holds the data and prints nothing

```
public class RideShare {
    private Driver[] drivers = new Driver[100];
    private int driverCount = 0;
    private Rider[] riders = new Rider[100];
    private int riderCount = 0;

    private static final double
        STARTING_RATING = 5.0;

    public Driver signUpDriver(String name, int zone)
    public Rider  signUpRider(String name)
    public User[] allUsers()
}
```

**Who does what**

**Console** — talks to the human.

**RideShare** — holds the accounts.

**User**, **Driver**, **Rider**, **Vehicle** — know their own facts.

One class, one job [9].

**RideShare** contains no `IO.` call at all — that is the whole discipline of this step.

ng Aiersilan | CSCI 2113

# `allUsers()` — one array, both kinds

- `signUpDriver` / `signUpRider`: create the object with the **starting rating** of **5.0** (a driver is marked available), store it in the matching array, and **return it**.
- `allUsers()`: a *new* `User[]` holding every driver, then every rider.

```
User[] all = new User[driverCount + riderCount];   // exactly this long
```

⚠ ✗ Not `new User[200]`. The length must be exactly the number of accounts, or the caller ends up looping over `null`s — the crash you spent Lab 3 learning to avoid.

# Listing users: one loop, two `toString()`s

```
User[] all = rideShare.allUsers();
for (int i = 0; i < all.length; i++) { IO.println(all[i]); }
```

```
Ada     | zone 3 | 5.0 | AVAILABLE   | null
Grace   | zone 2 | 5.0 | AVAILABLE   | null
Rider #3 Alan
```

The loop variable's type is **User**. The loop never asks whether it holds a driver or a rider — each object's own `toString()` runs.

**This is what the whole refactoring bought you.** Empty array → print `No users.` ⚠ At this point in the Activity a driver has *no* vehicle, so **Driver**'s Lab 3 `toString()` still pads its columns and prints `null` for the car. Part 3 of the assignment changes both.

# The Assignment

ng Aiersilan | CSCI 2113 | F

# What you are building

Continuing from the end of the Activity: **User**, **Driver**, **Rider**, **Vehicle**, **RideShare**, **Console**.

1. **Vehicle information** at driver sign-up — including a luxury flag.
2. **List vehicles**, derived from the drivers.
3. **Driver availability** — find one by id, toggle it, list who is free.
4. **Ride types** — an abstract **Ride** with three subclasses.

★ Due Sunday, September 27, 2026, by 11:59 PM — and Part **4** is where today's four mechanisms all meet.

ng Aiersilan | CSCI 2113

# Part 1: Vehicle learns about luxury

```
public Vehicle(String plate, int seats, boolean luxury)   // the real one
public Vehicle(String plate, int seats)   // this(plate, seats, false)
public Vehicle(String plate)              // this(plate, 4)
public boolean isLuxury()
```

```
DC-1001 (4 seater)
DC-1009 (4 seater, luxury)
```

Keep the Lab 3 constructors and chain them into the new one with `this(...)`. The seat rule stays where it was: a count below **1** is stored as **1**, written **once**, in the constructor everything else funnels into.

ng Aiersion / CSCI 2113

# Part 1: Console asks, RideShare builds

```
public Driver signUpDriver(String name, int zone, String plate,
                           int seats, boolean luxury)
```

## `signUpDriver`, in order

1. Make the **Vehicle** — plate, seats, luxury flag; a seat count below **1** becomes **1**.
2. Make the **Driver** — that zone, the starting rating, marked available, holding that vehicle.
3. Store it in the next free slot, and **return it**.

## What the user sees

```
Your choice: 1
Name: Ada
Zone: 3
Plate: DC-1001
Seats: 4
Luxury (y/n): n
Driver added
```

`Seats` is converted with `Integer.parseInt`. Exactly `y` means luxury; any other answer means not. ⚠ **Console** passes **five** values and ✗ never constructs the **Vehicle** itself.

# Part 2: `allVehicles()` is *derived*, not stored

```
public Vehicle[] allVehicles() { ... }
```

**RideShare** keeps **no** vehicle array. Each vehicle belongs to a driver, so the list is built by walking the drivers — in sign-up order.

```
Your choice: 4
DC-1001 (4 seater)
DC-1009 (4 seater, luxury)
DC-1006 (7 seater)
```

Every driver signed up through `signUpDriver` has a vehicle, so the array's length is exactly `driverCount`.

Empty → `No vehicles`. The listing shows the vehicles only — no driver information.

ng Aierdilan | CSCI 2113

# Part 3: the operator works by *id*

Names cannot identify a driver — two drivers may share one. So **Driver**'s `toString()` now leads with the id, in **Rider**'s style:

```
Driver #1 Ada | zone 3 | 5.0 | AVAILABLE | DC-1001 (4 seater)
```

- `findDriver(int id)` — that driver, or **null**. A *rider's* id also returns `null`: riders are not in the `drivers` array.
- `availableDrivers()` — a new `Driver[]` of exactly the available ones, in sign-up order.
- Option 5 reads `Driver id:`, flips the flag, and prints `Ada is now UNAVAILABLE`; a missing id prints `No driver with id 3`.
- Option 6 prints them one per line — or `No available drivers` when nobody is free.

⚠ ✖ No column padding. Print each line exactly as `toString()` produces it — unlike Lab 3's `printBoard()`.

# Part 4: three ways to be a ride

**Ride** is **abstract** and declares two abstract methods; each subclass implements both.

| Type | Class | Who can serve it | Fare |
|---|---|---|---|
| Standard | **StandardRide** | available; vehicle seats the party | **$2.50** + **$1.25**/mile |
| XL | **XLRide** | available; vehicle has $\geq$ **6** seats *and* seats the party | **$3.50** + **$1.75**/mile |
| Luxury | **LuxuryRide** | available; vehicle is a luxury vehicle *and* seats the party | twice standard, min **$15.00** |

```
public abstract boolean canBeServedBy(Driver driver);
public abstract double fare();
```

# Where the miles come from

```
private static final int[][] DISTANCE = {
    { 0,  5,  9, 12, 14},
    { 5,  0,  6,  8, 11},
    { 9,  6,  0,  4,  7},
    {12,  8,  4,  0,  5},
    {14, 11,  7,  5,  0}
};
public static int distance(int from, int to)
public static int zoneCount()
```

Copy both methods into **RideShare** as given.

**Two uses, don't mix them**

**Travel miles** of a ride:
`distance(pickupZone, dropoffZone)`

**Nearest** driver to the pickup:
smallest `distance(pickupZone, driverZone)`

Valid zones are **0** to `zoneCount()` − **1**. The table is symmetric with a zero diagonal, so a ride that starts and ends in the same zone travels **0** miles.

# The booking rule, in order

1. Keep the drivers for which `canBeServedBy(driver)` is `true`.
2. Of those, take the one **nearest the pickup zone**.
3. Tie → the **higher rating**.
4. Still tied → the one who **signed up first** (lower id).

Then: the assigned driver becomes unavailable, the ride takes the **next ride id**, and **RideShare** records it in a `Ride[]`. ⚠ A request that finds *no* driver is ✗ not booked: it prints `No driver available` and consumes no ride id.

ng Aiersilan | CSCI 2113

# Two constraints that shape your design

## 1. Differ by *overriding*

```
if (type.equals("XL")) { ... }   // no
if (r instanceof XLRide) { ... } // no
```

The three types must differ only through overridden methods.

## 2. A ride precedes its driver

`canBeServedBy` is asked *before* anyone is assigned. So a **Ride** has to exist with no driver yet — decide how you represent that.

Everything else is yours: fields, constructors, helpers, and how the work splits between **Ride** and its subclasses.

If you are reaching for `instanceof`, you have found a method that belongs **on the subclass** [2].

ng Aiersilan | CSU 2113

# The console script is fixed — match it exactly

```
Main Menu
(1) Sign up a driver
(2) Sign up a rider
(3) List users
(4) List vehicles
(5) Toggle a driver's availability
(6) List available drivers
(7) Request a ride
(8) List rides
(q) Quit
Your choice:
```

Option 7 reads, in this order: `Rider id:`, then the type after printing `(1) Standard / (2) XL / (3) Luxury` and `Ride type:`, then `Pickup zone:`, `Dropoff zone:`, `Party size:`. Option 8 prints every booked ride, in the order booked — or `No rides`.

```
Booked Ride #1 (Luxury): Ana with Grace | 14 miles | $40.00
No driver available
No rider with id 9
```

⚠ The write-up's prose shows a trailing period on two of these lines; the published transcript has none. Follow the transcript.

ng Aiersilan | 2113

# The published session *is* your test

| id | Account | Zone | Vehicle |
|---|---|---|---|
| 1 | Driver Ada | 3 | DC-1001, 4 |
| 2 | Driver Grace | 2 | DC-1009, 4, luxury |
| 3 | Rider Ana | — | — |
| 4 | Driver Linus | 0 | DC-1006, 7 |
| 5 | Rider Alan | — | — |

```
$ java Console < lab4-session-input.txt
```

On Windows:

```
Get-Content lab4-session-input.txt | java Console
```

Your output should match character for character.

💡 Option 3 lists drivers first, then riders — so the ids come out **1**, **2**, **4**, **3**, **5**. If yours reads **1**…**5** in order, ✗ your `allUsers()` is wrong.

# Five rides, and why each one comes out that way

| Ride | Type | Zones | Miles | Driver, and why | Fare |
|---|---|---|---|---|---|
| #1 | Luxury | 0→4 | 14 | Grace — the only luxury vehicle | $40.00 |
| #2 | Standard | 2→3 | 4 | Ada — zone 3 is 4 miles from the pickup; Linus is 9 | $7.50 |
| #3 | XL | 1→4 | 11 | Linus — the only vehicle with 6 seats or more | $22.75 |
| #4 | Standard | 0→1 | 5 | Ada — the only driver available again | $8.75 |
| #5 | Luxury | 2→2 | 0 | Grace | $15.00 |

#1: $2 \times (2.50 + 1.25 \times 14) = \$40.00$. #3: $3.50 + 1.75 \times 11 = \$22.75$. #5: $2 \times 2.50 = \$5.00$, raised to the $15.00 minimum.

⚠ Between #3 and #4 a request finds nobody — all three drivers are out. It prints `No driver available`, and the next booked ride is #4, ✗ not #5.

# Part 5: the checklist to submit against

```
abstract class User
  User(String name, double rating)
  getId  getName  getRating  setRating

class Driver extends User
  Driver(name, zone, available, rating)
  Driver(name, zone, available, rating,
         vehicle)
  getZone  isAvailable  setAvailable
  getVehicle  toString  equals(Driver)

class Rider extends User
  Rider(String name, double rating)
  toString
```

```
class Vehicle
  Vehicle(plate, seats, luxury)
  Vehicle(plate, seats)   Vehicle(plate)
  getPlate  getSeats  isLuxury
  toString  equals(Vehicle)

class RideShare
  signUpDriver(name, zone, plate,
               seats, luxury)
  signUpRider  allUsers  allVehicles
  findDriver  availableDrivers
  static distance(int from, int to)

abstract class Ride
  canBeServedBy(Driver)  fare  toString
```

Extra helper methods and constructors are welcome. All `.java` files in one directory, no packages.

# Run the checker, then submit

```
java -jar check-lab4.jar          # put the jar beside your .java files

tar -czf lab4-assignment.tgz *.java                    # Linux / Windows
tar --no-mac-metadata -czf lab4-assignment.tgz *.java  # macOS
```

- Upload `lab4-assignment.tgz` at `submit.cs.seas.gwu.edu` → *CSCI 2113: Software-Engineering* → *Lab 4 - Assignment*.
- Cannot log in? Reset the password first; if that fails, contact an instructor.

★ Due Sunday, September 27, 2026, by 11:59 PM. Grading assumes well-formed input.

ng Aiersilan | CSCI 2113

# Wrap-up

ng Aiersilan | CSCI 2113 | F

# Habits, and the traps they avoid

| Do | Don't |
| --- | --- |
| ✔ `@Override` on every override. | ✘ Trust a method name you typed once. |
| ✔ `getName()` from a subclass. | ✘ Reach into a parent's private field. |
| ✔ Right-size every returned array. | ✘ Return `new User[100]`. |
| ✔ `.equals(...)` for menu input. | ✘ Ask `instanceof` what to do. |

💡 Every "don't" on the right is a question the type system could have answered for you.

# Takeaways

1. ★ **Every constructor chains upward.** A subclass constructor finishes an object the parent started.
2. ★ **The declared type picks the method; the object picks the body.** That one split is polymorphism.
3. ★ **Abstract classes share code; interfaces share a contract.** **User** is the first; **Describable** is the second.

**In one sentence**

Say the shared thing once, in a parent — and let each subclass answer for itself.

ng Aielian | CSCI 2113

# Acknowledgement

ng Aiersilan | CSCI 2113 | F

# Acknowledgement

This lab builds on the initial materials developed by **Timothy Kim**.

We gratefully acknowledge his contribution.

ng Aiersilan | CSCI 2113

# References


[1] J. Bloch. *Effective Java*. Addison-Wesley Professional, 3rd edition, 2018. Items 20 and 40.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Chapter 1.

[3] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman. *The Java Language Specification, Java SE 25 Edition*. Oracle America, Inc., 2025. https://docs.oracle.com/javase/specs/jls/se25/html/.

[4] B. H. Liskov and J. M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994. DOI 10.1145/197320.197383.

[5] OpenJDK. JEP 512: Compact Source Files and Instance Main Methods. OpenJDK JDK Enhancement Proposal, delivered in JDK 25, 2025. https://openjdk.org/jeps/512.

[6] OpenJDK. JEP 513: Flexible Constructor Bodies. OpenJDK JDK Enhancement Proposal, delivered in JDK 25, 2025. https://openjdk.org/jeps/513.

[7] Oracle. Class java.lang.IO. Java SE 25 API Specification, 2025. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/IO.html.

[8] Oracle. Flexible Constructor Bodies. Java SE 25 Language Updates, 2025. https://docs.oracle.com/en/java/javase/25/language/flexible-constructor-bodies.html.

[9] R. Wirfs-Brock and A. McKean. *Object Design: Roles, Responsibilities, and Collaborations*. Addison-Wesley Professional, 2002. Source of the role stereotypes.

# Thank you!

**Questions?**

# Backup: why option 3 prints 1, 2, 4, 3, 5

```
Driver #1 Ada   | zone 3 | 5.0 | AVAILABLE | DC-1001 (4 seater)
Driver #2 Grace | zone 2 | 5.0 | AVAILABLE | DC-1009 (4 seater, luxury)
Driver #4 Linus | zone 0 | 5.0 | AVAILABLE | DC-1006 (7 seater)
Rider #3 Ana
Rider #5 Alan
```

(Shown aligned here for reading only — your program prints no padding.)

Two different orders are at work. Ids are handed out by **User** in **sign-up order**, across both classes. `allUsers()` copies **every driver first**, then every rider. The two orders disagree, and that disagreement is the evidence that one counter really does serve the whole hierarchy.

ng Aiersten / CSCI 2113

# Backup: `equals(Driver)` is still an *overload*

```
Driver a = new Driver("Ada", 3, true, 5.0);
Driver b = new Driver("Ada", 3, true, 4.0);
IO.println(a.equals(b));        // true  -> your equals(Driver)
Object o = b;
IO.println(a.equals(o));        // false -> Object.equals(Object)
```

Your Lab **3** rule — same name, same zone — makes those two equal even though the ratings differ. And because `equals(Driver other)` takes a **Driver**, it *adds* a method rather than replacing **Object**'s. Which one runs is decided by the declared type of the argument, at compile time — the same rule as the **Describable** array.

⚠ Writing `@Override` above it would ✗ fail to compile, which is exactly the annotation doing its job. Proper overriding means `equals(Object)`; that is beyond this lab [1].

# Backup: what else an interface may hold

```
public interface Describable {
    String describe();                          // implicitly public abstract
    default String shout() {                    // a body, inherited by all
        return describe().toUpperCase();
    }
    static Describable of(String s) { ... }     // called on the interface itself
    int MAX = 100;                              // implicitly public static final
}
```

Interfaces have been able to carry **default** and **static** methods since Java **8**, and `private` helper methods since Java **9**. What they still cannot have is instance fields — no per-object state.

⚠ Lab **4** needs none of this. It is here so the answer to "can an interface have a body?" is *yes, but that is not what we are doing today.*

ng Aiersilan / CSCI 2113

# Backup: the error messages, in one place

```
constructor Parent in class Parent cannot be applied to given types;
        -> the inserted super() found no no-argument constructor

method does not override or implement a method from a supertype
        -> @Override on a name that matches nothing above it

X is not abstract and does not override abstract method m() in Y
        -> a concrete class still owes an implementation

Parent is abstract; cannot be instantiated
        -> you may use the type, not the constructor

name has private access in User
        -> a subclass is not an exception to private

cannot find symbol / location: interface Describable
        -> the declared type, not the object, decides the menu
```

Six errors, six lessons. Read the message, then the line it names — and remember the cause is often in the *other* file.

ng Aiersilan | CSCI 2113